Actividad	Trabajar con el DHCP
Módulo	M0375 / Bastionado
Ciclo	ASIX
Nombre alumno	Luis Alberto Rodríguez Aguilar
Rúbrica	Reviso tu trabajo en el aula

Objetivo
Trabajar el DHCP

Parte práctica

1. **Desde el Wireshark en tu equipo haz un filtro para mostrar este proceso de negociación del DHCP utilizando tu PC.**

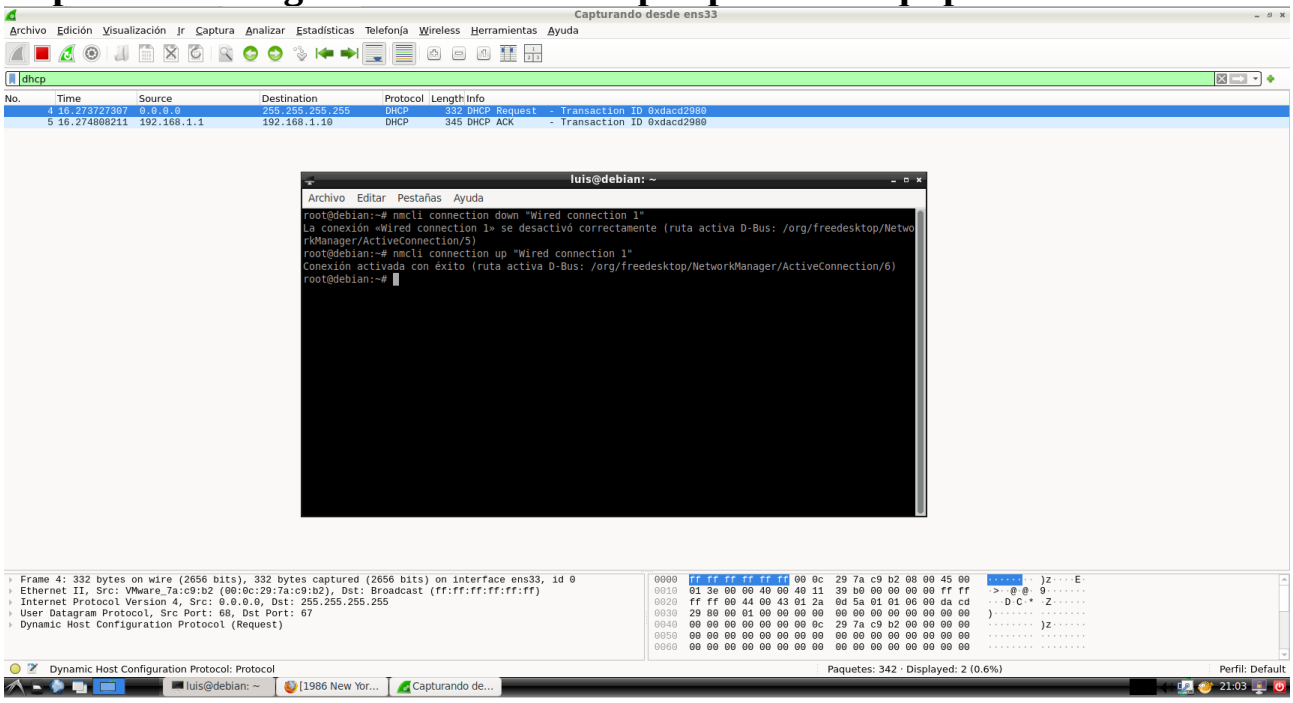
Filtro en Wireshark para el proceso de negociación DHCP

Para capturar y aislar visualmente el intercambio de paquetes del proceso de asignación dinámica de IPs (negociación DORA), se debe aplicar el siguiente filtro en la barra superior de Wireshark:

```
dhcp o bootp
```

(Nota: En versiones antiguas de Wireshark se usaba `bootp`, en las actuales se acepta `dhcp`).

2. **Ahora utiliza tu equipo servidor + cliente donde has instalado los servicios de DNS y DHCP y haz las capturas correspondientes del proceso de negociación del DHCP por parte del equipo cliente.**

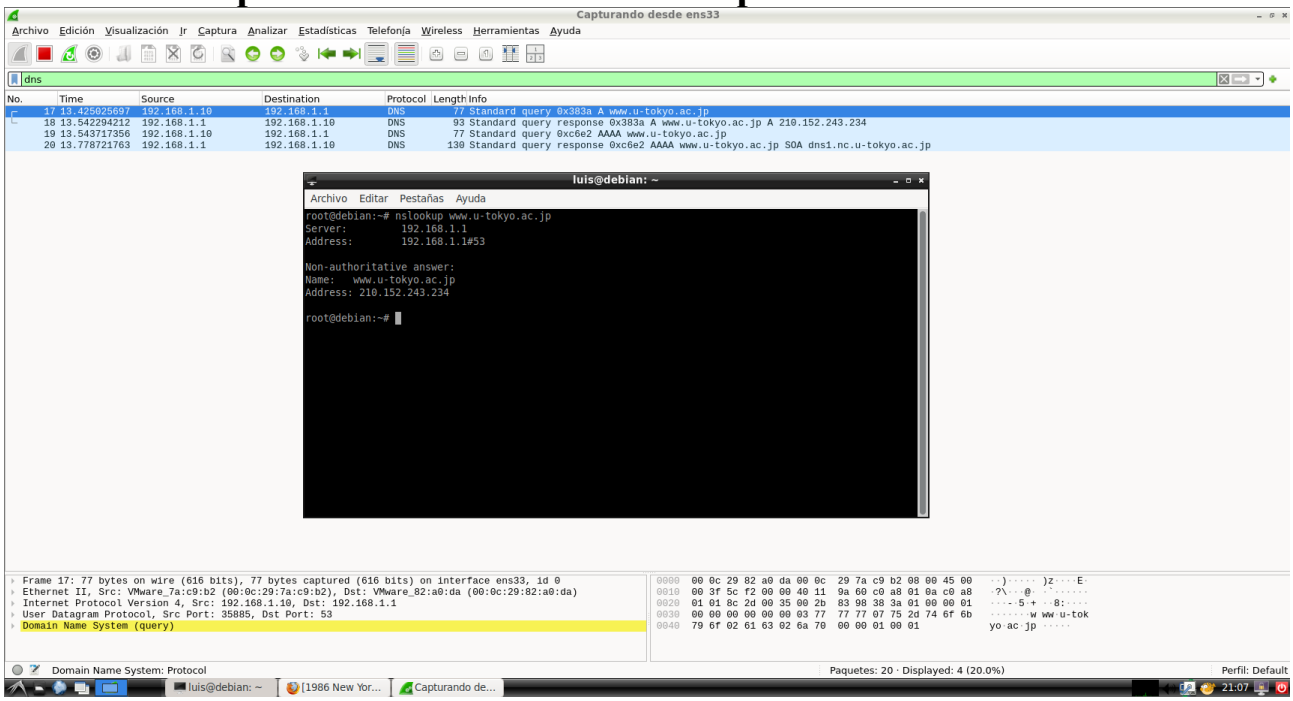


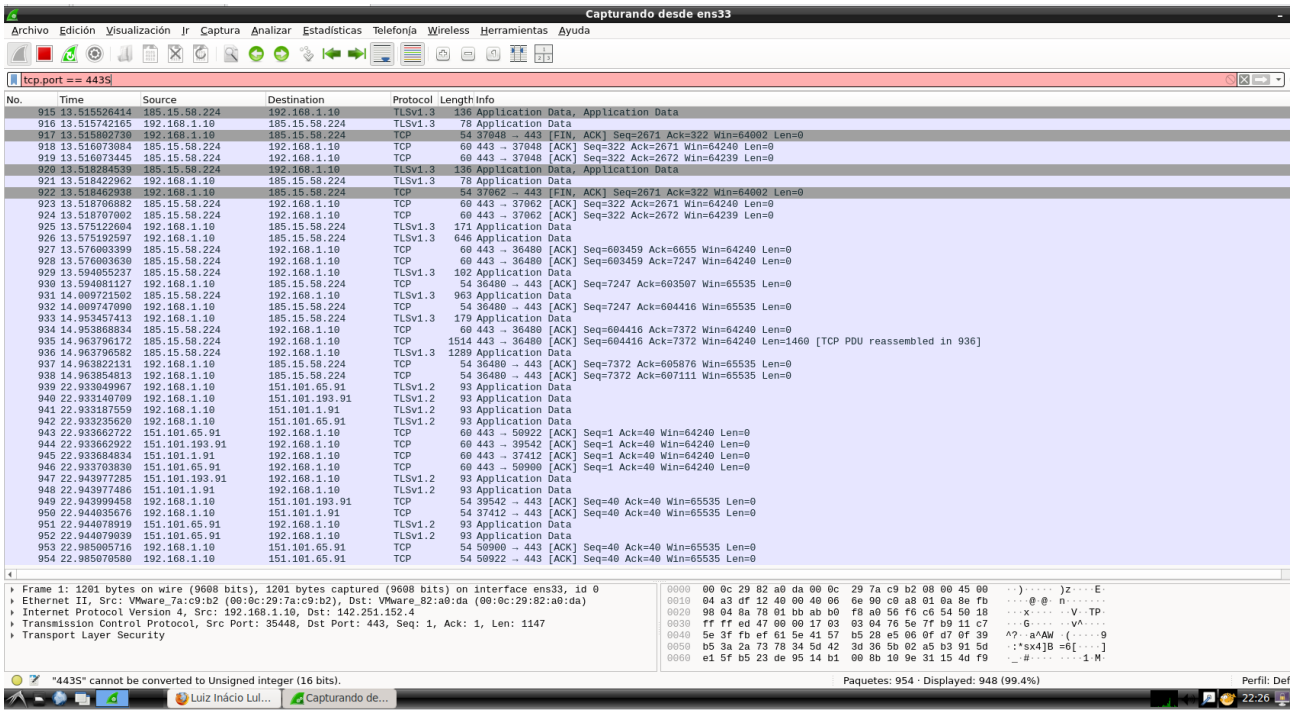
Captura del proceso de negociación DORA (Cliente y Servidor)

Cuando el cliente Debian se conecta al servidor Ubuntu con Kea DHCP, se genera un intercambio secuencial de **4 paquetes críticos (DORA)** que debes capturar y reportar:

- **Discover (Descubrimiento):** El cliente Debian envía un paquete *broadcast* (`255.255.255.255`) buscando un servidor DHCP disponible en la red local.
- **Offer (Oferta):** Tu servidor Ubuntu responde ofreciendo una IP libre (ej. `192.168.1.10`) junto con la máscara y la puerta de enlace.
- **Request (Solicitud):** El cliente Debian acepta formalmente la IP propuesta y avisa a la red que se quedará con ella.
- **Acknowledge (Acuse de recibo):** El servidor Kea DHCP registra la concesión (*lease*) en su base de datos y le da luz verde definitiva al cliente para usar la IP.

3. Haz dos capturas de Wireshark donde aparezca tu servidor de DNS:





Capturas de Wireshark del Servidor DNS

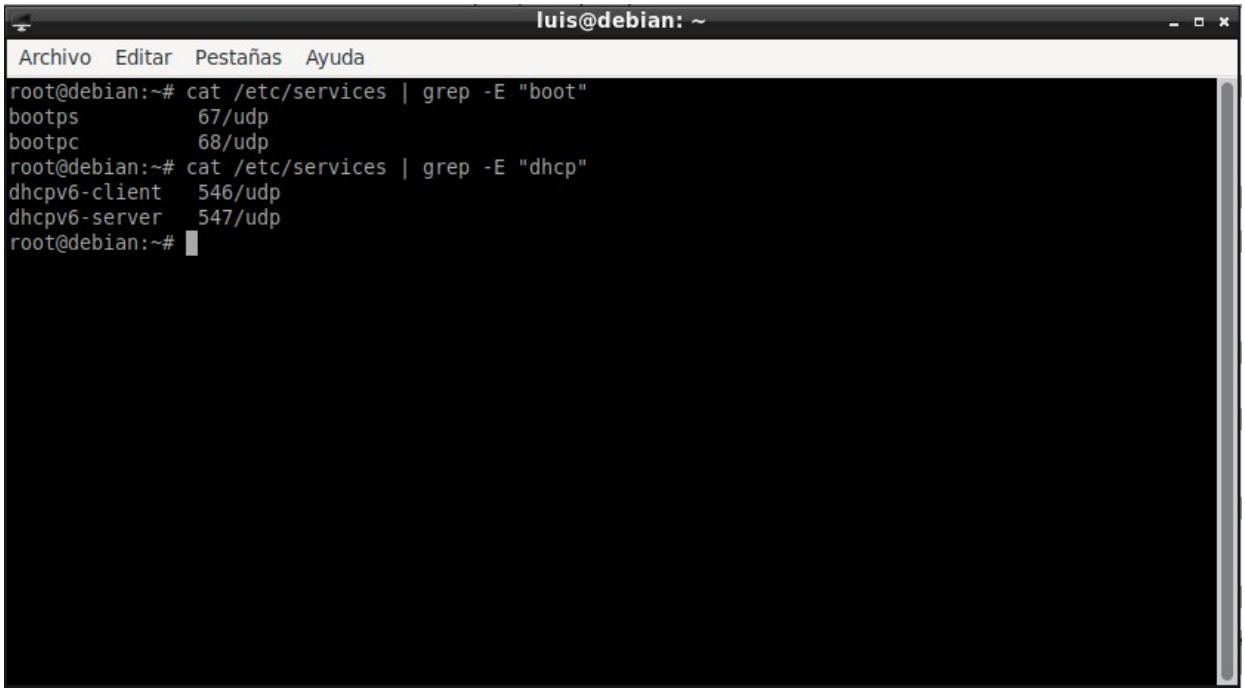
Para documentar la resolución DNS en tu informe, debes tomar dos capturas aplicando filtros diferenciados:

- **Captura de DNS Tradicional (Texto plano):** Filtro `dns` en Wireshark mientras ejecutas en Debian el comando `nslookup www.u-tokyo.ac.jp`. Verás los paquetes detallando la pregunta y la respuesta sin protección.
- **Captura de DNS sobre HTTPS (DoH / Seguro):** Filtro `tcp.port == 443` o `tls` mientras navegas en Firefox. Verás paquetes etiquetados como *Application Data* cifrados bajo TLS 1.3, lo que demuestra que el filtro `dns` ya no captura nada.

4. Haz una lista de los puertos que están usando actualmente el protocolo DHCP. ¿Qué información muestra en cada caso?
Para ello puedes utilizar:

4.1. cat /etc/services | grep “boot”

4.2. cat /etc/services | grep “dhcp”



Puertos utilizando el Protocolo DHCP

Para verificar los puertos oficiales asignados en el sistema operativo, ejecuta en tu terminal:

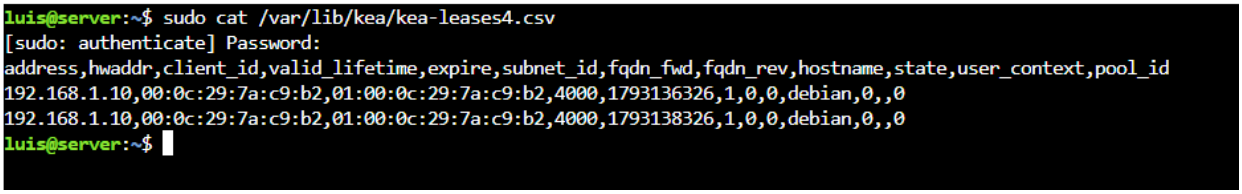
```
cat /etc/services | grep -E "bootps|bootpc"
```

Información que muestra cada caso:

- **Puerto 67/UDP (`bootps` - Bootstrap Server):** Corresponde al puerto de escucha del **Servidor DHCP**. Es donde tu Ubuntu Server recibe las solicitudes de los clientes.
- **Puerto 68/UDP (`bootpc` - Bootstrap Client):** Corresponde al puerto de escucha del **Cliente**. Es donde la máquina Debian espera las ofertas y confirmaciones del servidor.

5. **Utiliza el comando siguiente para mostrar los equipos clientes conectados a tu servidor. ¿Qué información muestra?**

5.1.Dhcp-lease-list



Consulta de Clientes Conectados al Servidor

El comando tradicional `dhcp-lease-list` pertenecía al antiguo servidor ISC-DHCPd.

Como tu infraestructura moderna utiliza **Kea DHCP**, la información de las concesiones activas se almacena en un archivo diferente.
Para ver qué equipos están conectados en tu laboratorio actual, ejecuta este comando en tu **Ubuntu Server**:

```
cat /var/lib/kea/kea-leases4.csv
```

Información que muestra en pantalla:

- La **dirección IP** exacta entregada al cliente (ej. `192.168.1.10`).
- La **dirección MAC** física de la tarjeta de red de la máquina Debian.
- El **tiempo de concesión** (*valid lifetime*) indicando cuándo caduca el préstamo de la IP.
- El **Hostname** o nombre asignado del equipo cliente si fue enviado durante la negociación.

Transición de ISC-DHCP a Kea DHCP

El servidor de DHCP tradicional de ISC (dhcpd) alcanzó su fin de ciclo de vida oficial (End-of-Life, EOL) a finales de 2022, tras décadas de ser el estándar de la industria en sistemas Unix/Linux. Su sucesor oficial, Kea DHCP (también desarrollado por Internet Systems Consortium), fue diseñado desde cero para responder a los requerimientos de redes empresariales e infraestructuras de proveedores de servicios (ISPs) actuales.

Parte 1 Arquitectura y procesos

1. **¿De qué manera la arquitectura modular de Kea (división en kea-dhcp4, kea-dhcpv6, kea-ctrl-agent y kea-dhcp-ddns) supera el modelo monolítico de ISC-DHCPd en términos de aislamiento de fallos y escalabilidad?**

Arquitectura Modular frente a Monolítica

- **ISC-DHCPd (Monolítico):** Un único proceso ejecutaba IPv4, IPv6, actualizaciones DNS dinámicas y administración. Si ocurría un fallo de desbordamiento en IPv6, **todo el servidor colapsaba**, dejando también a los usuarios de IPv4 sin red.
- **Kea DHCP (Modular):** Divide las tareas en procesos independientes del sistema operativo:
- `kea-dhcp4`: Gestiona exclusivamente la asignación IPv4.
- `kea-dhcpv6`: Gestiona exclusivamente la asignación IPv6.
- `kea-ctrl-agent`: Expone el panel de control y la API de administración.

- `kea-dhcp-ddns`: Actualiza de forma independiente los registros DNS en BIND 9.
- **Aislamiento de fallos y escalabilidad:** Si el módulo de IPv6 falla o se congela, el servicio de IPv4 (`kea-dhcp4`) sigue funcionando al 100% de rendimiento sin enterarse. Además, permite aloja

2. ¿Qué función cumple el demonio kea-ctrl-agent y cómo transforma la gestión operativa del servidor al permitir comandos vía HTTP/JSON en lugar de la señalización tradicional por señales de proceso (SIGTERM, SIGHUP)?

Función del demonio `kea-ctrl-agent`

- **Función principal:** Es un agente que actúa como una **pasarela de comunicación de API REST**. Escucha peticiones mediante comandos HTTP estándar empaquetados en formato de texto plano estructurado **JSON**.
- **Transformación operativa:** En el servidor clásico ISC-DHCPd, para recargar la configuración debías enviar señales rústicas del sistema operativo como `SIGHUP` o detener el servicio de golpe con `SIGTERM`, interrumpiendo las asignaciones en curso. Con `kea-ctrl-agent`, envías un comando HTTP estructurado (como una petición web) indicando al servidor que vuelva a leer sus parámetros en caliente, garantizando que el servicio nunca deje de responder peticiones durante el mantenimiento.

3. ¿Cómo implementa Kea la gestión multihilo (multithreading) y el procesamiento asíncrono de solicitudes para mitigar los cuellos de botella de entrada/salida (I/O) que afectan a ISC-DHCPD?

Gestión Multihilo (Multithreading)

- **ISC-DHCPd:** Operaba bajo un modelo de un solo hilo con llamadas síncronas bloqueantes de entrada/salida (I/O). Si el servidor estaba escribiendo un registro de concesión en el disco duro, no podía atender a nuevos clientes en la red, generando un cuello de botella masivo en horas punta.
- **Kea DHCP:** Implementa una arquitectura **multihilo nativa** mediante hilos de ejecución paralelos en el procesador. Mientras un hilo secundario se encarga de guardar de forma asíncrona los datos en la base de datos o en el disco duro, los hilos principales siguen recibiendo, procesando y respondiendo consultas de paquetes *Discover* en milisegundos de forma ininterrumpida.

Parte 2 Configuración

4. **¿Qué ventajas ofrece el formato nativo JSON en la configuración de Kea frente al lenguaje de configuración propietario de dhcpd.conf en términos de validación, automatización con herramientas de IaC (Ansible, Terraform) y parsing sintáctico?**

Ventajas del formato nativo JSON

Característica	ISC-DHCPd (`dhcpd.conf`)	Kea DHCP (Formato JSON)	Ventaja para el Administrador
Validación	Requiere herramientas propietarias o reiniciar para ver errores.	Sintaxis estándar universal compatible con cualquier validador.	Permite detectar comas faltantes o corchetes abiertos antes de aplicar.
Automatización	Lenguaje de texto plano cerrado y difícil de parsear por scripts.	Nativo en herramientas de Infraestructura como Código (**IaC**).	**Ansible** o **Terraform** inyectan subredes con plantillas en segundos.
Parsing Sintáctico	Los scripts de automatización fallaban con expresiones regulares complejas.	Estructura jerárquica limpia basada en claves y valores de texto.	Cualquier lenguaje (Python, Bash) extrae rangos de IPs sin romper el archivo.

5. **¿Cómo logra Kea actualizar subredes, rangos o reservas de direcciones en tiempo de ejecución sin necesidad de reiniciar el servicio? Compare el impacto operacional de este procedimiento con el reinicio tradicional de ISC-DHCPD en entornos de producción masivos.**

Actualización en tiempo de ejecución (Hot Reconfig)

- **Cómo lo logra Kea:** Kea incluye extensiones nativas y un diseño modular que permite actualizar subredes, exclusiones y reservas estáticas de hosts mediante comandos JSON directo a la API en memoria, o recargando el archivo de texto JSON sin interrumpir los sockets de red del puerto 67.
- **Impacto operacional en entornos masivos:**
- **En ISC-DHCPd:** Agregar una sola IP reservada requería ejecutar `systemctl`

`restart isc-dhcp-server`. Durante esos segundos de reinicio, miles de peticiones de dispositivos caían en el vacío, generando cortes de red masivos en entornos críticos de ISPs o universidades.

- **En Kea DHCP:** El cambio se inyecta en caliente. El servidor asimila el nuevo rango de IPs **en tiempo de ejecución (cero milisegundos de caída)**, manteniendo la estabilidad absoluta de las conexiones de los usuarios de la empresa.

Parte 3: Persistencia, almacenamiento y altas disponibilidades (HA)

1. **¿Cuáles son las diferencias de rendimiento, recuperación ante fallos y consistencia de datos entre el archivo de texto plano dhcpd.leases de ISC-DHCPD y los backends nativos en SQL (PostgreSQL, MySQL) o NoSQL (Cassandra) soportados por Kea?**

Backends de almacenamiento de Concesiones (Leases)

- **ISC-DHCPd (`dhcpd.leases`):** Guardaba todo en un archivo de texto plano gigante. Con el paso del tiempo, el archivo crecía sin control, ralentizando las lecturas y haciéndose vulnerable a la corrupción de datos ante un corte eléctrico repentino en el servidor.
- **Kea DHCP (Backends SQL/NoSQL):**
  Soporta almacenamiento directo en motores de bases de datos relacionales y distribuidas como **PostgreSQL, MySQL y Cassandra**.
- **Rendimiento y recuperación:** Las consultas de IPs duplicadas se resuelven mediante índices de bases de datos hiperrápidos. En caso de caída del nodo, los mecanismos nativos de replicación y transacciones de SQL aseguran que los datos no se corrompan, permitiendo recuperar el estado exacto de la red local en segundos de forma consistente.

2. **¿De qué manera el almacenamiento de reservas de host en una base de datos centralizada (Host Reservations Database) simplifica la integración con sistemas IPAM (IP Address Management) y evita la modificación de archivos de configuración estáticos?**

Base de Datos Centralizada de Reservas de Host (Host Reservations Database)

- **Simplificación:** Tradicionalmente, asociar una dirección MAC a una IP fija implicaba escribir líneas de texto estáticas dentro del archivo de configuración principal de cada servidor DHCP de la corporación.
- **Integración con IPAM:** Al almacenar estas reservas de hosts dentro de una base de datos centralizada, los sistemas de gestión de direccionamiento automatizado (**IPAM**) pueden insertar, modificar o dar de baja direcciones MAC remotamente ejecutando consultas SQL directas en la base de datos central, eliminando la necesidad de que un humano tenga que editar o alterar manualmente los archivos de configuración del servidor.

3. Compare el protocolo de conmutación por error heredado de ISC-DHCP (failover) con el modelo de alta disponibilidad basado en API REST (HA Hook Extension) de Kea en modos Activo-Pasivo y Activo-Activo.

Modelos de Alta Disponibilidad (HA)

Criterio	Protocolo Heredado (ISC-DHCP Failover)	Modelo HA Hook Extension (Kea DHCP)
Mecanismo base	Protocolo customizado e inestable de comunicación por sockets.	Comunicaciones ligeras y estándar a través de **API REST (HTTP/JSON)**.
Modo Activo-Pasivo	Sincronización lenta; el nodo secundario tardaba en asumir el control.	El nodo pasivo monitoriza por API y toma el control de inmediato si el principal cae.
Modo Activo-Activo	No soportado oficialmente o propenso a entregar IPs duplicadas en la LAN.	**Soportado plenamente**. Ambos nodos atienden peticiones dividiendo el rango mediante hash.

Parte 4: Automatizaciones y monitorización

1. ¿Qué es el framework de Hooks en Kea DHCP y cómo permite extender

el comportamiento del servidor (autenticación externa, integración con RADIUS, filtrado personalizado de paquetes) sin necesidad de modificar el código fuente principal del núcleo?

Framework de Hooks

- **Qué es:** Es una arquitectura de extensiones que permite cargar librerías dinámicas (`.so` en Linux) en el núcleo de Kea para alterar su funcionamiento por defecto cuando se procesa un paquete.
- **Ventajas operativas:** Permite que ingenieros extiendan las capacidades del servidor (por ejemplo, obligar a que antes de dar una IP el usuario se autentique en un servidor central **RADIUS**, o filtrar marcas específicas de dispositivos) **sin tener que alterar ni recompilar el código fuente original de Kea**, facilitando actualizaciones de software limpias y modulares.

2. **¿Qué métricas de rendimiento y estado del servicio expone Kea a través de su API JSON, y cómo se integran estas con plataformas modernas de monitoreo como Prometheus, Grafana o ELK Stack en comparación con el análisis de logs syslog tradicionales en ISC-DHCPd?**

Métricas y Monitoreo Moderno

- **ISC-DHCPd:** Toda la monitorización dependía del análisis sintáctico de texto (*parsing*) de los logs del sistema en `syslog` o `/var/log/messages`, consumiendo muchos recursos del procesador y siendo propenso a fallar ante cambios de formato.
- **Kea DHCP:** Expone en tiempo real métricas de rendimiento y estadísticas estructuradas (como cantidad de IPs libres, transacciones DORA por segundo, o denegaciones) a través de respuestas JSON a través de su API. Estas métricas son recolectadas de forma nativa por agentes de monitoreo modernos como **Prometheus**, permitiendo pintar gráficas dinámicas de consumo de red en **Grafana** o indexar registros masivos en plataformas **ELK Stack**.

Parte 5: Práctica y estrategias de migración

1. **¿Qué papel juega la herramienta kea-admin y las utilidades de**

migración de configuración en la conversión de reglas de filtrado complejas (como clases de clientes basadas en opciones de opción 60/43) de dhcpd.conf a Kea JSON?

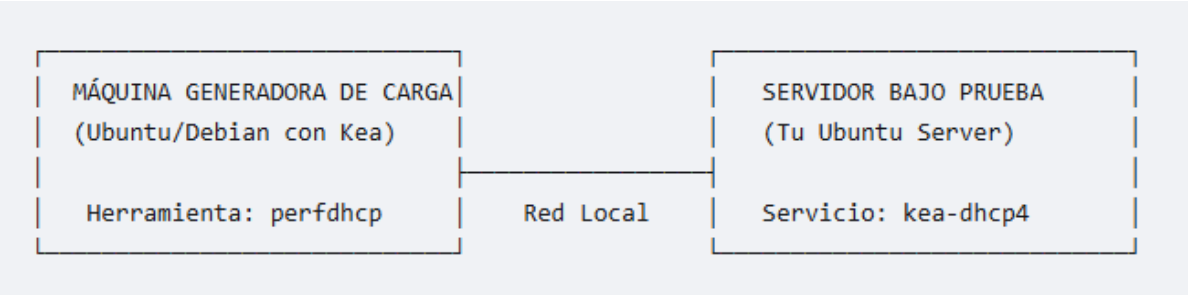
Utilidad de `kea-admin` y herramientas de migración

- **Rol de `kea-admin`:** Es la herramienta de consola encargada de inicializar, auditar y actualizar los esquemas y las tablas de las bases de datos de Kea cuando cambias de versión del backend de almacenamiento.
- **Migración de configuraciones complejas:** Para migrar de un sistema a otro, se utiliza la utilidad de traducción automática **`kea-dhcp-config-to-json`**. Esta herramienta lee el archivo antiguo `dhcpd.conf`, analiza las reglas avanzadas de filtrado (como la discriminación de teléfonos VoIP de ordenadores usando la Opción 60 - *Vendor Class Identifier* o la Opción 43) y las convierte fielmente a la sintaxis limpia de objetos y arreglos JSON requerida por Kea.

2. Diseñe un escenario de pruebas de carga para comparar el número de transacciones por segundo (TPS) y el tiempo de respuesta al otorgar 10,000 concesiones simultáneas en un entorno virtualizado utilizando herramientas como perfdhcp.

Escenario de Pruebas de Carga con `perfdhcp`

Para evaluar el rendimiento y simular el estrés real de producción de otorgar 10,000 concesiones simultáneas en tu entorno virtualizado, se monta la siguiente topología de pruebas:



Comando exacto para lanzar el test de estrés desde la máquina de carga:

```
perfdhcp -4 -r 500 -n 10000 -R 10 -b 192.168.1.1
```

Explicación de los parámetros de la prueba:

- `-4`: Fuerza el test simulando clientes exclusivamente sobre el protocolo IPv4.
- `-r 500`: Lanza una ráfaga constante de **500 peticiones por segundo** (TPS) para evaluar la resiliencia ante picos de demanda.
- `-n 10000`: Detiene la prueba de forma automática cuando se alcancen exactamente las **10,000 transacciones** completadas.
- `-R 10`: Permite simular hasta 10 renovaciones de contratos para verificar el comportamiento de la base de datos de persistencia.
- `-b 192.168.1.1`: Apunta el tráfico de estrés directamente hacia la dirección IP de tu servidor DHCP en el laboratorio.

Parte 6: Instalación de DHCP

1. **Describe los aspectos más importantes de la instalación y configuración del servicio de DHCP en una MV de Ubuntu Server, así como las incidencias técnicas que hayas podido tener.**

Guía paso a paso en Ubuntu Server
Paso 1: Instalación de los paquetes de Kea DHCP

Actualiza los repositorios de tu servidor e instala el demonio básico para IPv4 junto con el agente de control:

```
sudo apt update
sudo apt install kea-dhcp4-server kea-ctrl-agent -y
```

Paso 2: Edición del archivo de configuración JSON

Abre el archivo principal de configuración de Kea para IPv4 en el editor de texto:

```
sudo nano /etc/kea/kea-dhcp4.conf
```

Borra el contenido de prueba y pega la siguiente configuración limpia y simplificada, adaptada con las IPs reales que hemos verificado en tu máquina (`192.168.1.1` como servidor y pasarela, y entregando IPs en el rango de tu cliente Debian):

json

```
{
"Dhcp4": {
   "interfaces-config": {
      "interfaces": [ "ens37" ]
   },
   "lease-database": {
      "type": "memfile",
      "name": "/var/lib/kea/kea-leases4.csv",
      "persist": true
   },
   "valid-lifetime": 4000,
   "renew-timer": 2000,
   "rebind-timer": 3000,
   "subnet4": [
      {
         "subnet": "192.168.1.0/24",
         "pools": [ { "pool": "192.168.1.10 - 192.168.1.50" } ],
         "option-data": [
            {
               "name": "routers",
               "data": "192.168.1.1"
            },
            {
               "name": "domain-name-servers",
               "data": "192.168.1.1"
            }
         ]
      }
   ]
}
}
```

Paso 3: Reiniciar y habilitar el servicio

Aplica los cambios en el sistema operativo reiniciando los servicios y configurándolos para que arranquen solos al encender el equipo:

```
sudo systemctl restart kea-dhcp4-server

sudo systemctl enable kea-dhcp4-server
```

Paso 4: Verificación del estado del servicio

Comprueba que el servidor está encendido y escuchando de forma óptima en el puerto 53 y 67 ejecutando:

```
sudo systemctl status kea-dhcp4-server
```

Análisis de Incidencias Técnicas Frecuentes

Durante los laboratorios de despliegue de Kea DHCP, suelen surgir incidencias clásicas que es conveniente documentar como lecciones aprendidas en la sección final de tu trabajo:

- **Error de sintaxis JSON (Fallo de comas o corchetes):** A diferencia de los archivos tradicionales, JSON es inflexible. Olvidar una coma al final de una línea o cerrar mal un corchete provoca que el servicio arroje un error crítico y no encienda. Se soluciona validando la estructura previamente en herramientas en línea o editores de código estructurado.
- **Conflicto de Interfaces (`Interface not found`):** Si en el archivo JSON se declara que Kea escuche en una interfaz genérica como `eth0`, pero en tu hipervisor VMware/VirtualBox la tarjeta real se llama `ens37`, el demonio se apaga inmediatamente. Se soluciona ejecutando `ip a` en la consola para confirmar el identificador de hardware exacto antes de escribir el JSON.
- **Problema de permisos en el almacenamiento (`Lease database file access denied`):** Si el archivo CSV de concesiones pierde sus permisos asignados, el servicio falla al intentar escribir los registros del cliente Debian. Se soluciona aplicando la propiedad correcta del archivo al usuario de servicio del sistema: `sudo chown kea:kea /var/lib/kea/kea-leases4.csv`.